

27

TUESDAY  
FEBRUARY

WK 09 (058-307)

[illegible]

Text Area :- For article, paragraph, passage by the user.

<label for = "feedback"> Please provide feedback:  
</label>

< text area id = "feedback" > < / text area >

default size  $\rightarrow$  2 rows, 10 columns

can be changed:

placeholder can also be added.  $rows = " "$   $cols = " "$

placeholder = "Write Your feedback here"

CSSS

## Casading Style Sheets

HTM 2.2 skeleton, CSS for the skin

It is a language that is used to describe the style of a document.

2-y. text  $\rightarrow$  font, color, bg color, buttons style  $\downarrow$   
properties

HTML is content CSS is ~~for~~ style

But CSP is bigger than HTML

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |    |
|    |    |    |    | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 |
| 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 |    |

28

WEDNESDAY  
FEBRUARY

## Basic Format of CSS :-

u.g.

```
hl {  
  color: red;  
}
```

selector

property

value

### How to Include Styles :-

### 1. Inline Styles

Writing style directly inline on each element.

e.g. `<h1 style="color: red">Hello </h1>`

~~But~~ But this is not feasible way

2. Using `<style>` tag

Style  $\phi$  is added using the  $\langle \text{style} \rangle$  element in a the same document

`<style>`  
`h1 {`  
`color: red;`  
`}`  
`</style>`

→ This will apply to all h1.

This tag is added inside ~~the~~ head tag. This is ~~not~~ also not a flexible way when multiple html pages are created.

2018



This is very feasible way and will be always used ~~more~~. Because multiple HTML pages can be linked ~~to~~ with one CSS sheet. This is most valid and practicle way.

### Linking HTML2 with CSS :-

`<head>`

`</head>`

`link rel = "stylesheet" href = "style.css">`

↓

relation

↓

file name or path

Color Property :-

Used to set the color of foreground.

ways to write

color: red; → simple color name

color: #ff0000; → color code

color: rgb(214, 122, 127); → rgb color

color: hsl (30, 100%, 50%)

color: hsla(30, 100%, 50%, .3)

color: white (1.5708 rad 20° 10' 10")



01

THURSDAY  
MARCH

MARCH 2018

|    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|
| M  | T  | W  | T  | F  | S  | M  | T  | W  | T  | F  | S  | S  |
|    |    |    |    |    |    | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |    |    |

WK 09 (060-305)

8 am Background color property :-

9 am Used to set the color of background

10 am e.g. background-color: purple;

11 am only background: purple; will do ~~not~~ work  
by background has ~~no~~ other functions  
also ~~no~~ its better way to  
use background-color

12 noon

1 pm Color Systems :-

2 pm Named colors

Each browser can recognize around 140-150  
~~named~~ named colors.

3 pm

e.g. red, white, teal, purple, pink, etc.

4 pm

RGB

5 pm Red Green Blue (0-255)

6 pm

Red channel 0 ————— 255

Green Channel 0 ————— 255

Blue Channel 0 ————— 255

red → rgb(255, 0, 0)

black → rgb(0, 0, 0)

blue → rgb(0, 0, 255)

yellow → rgb(255, 255, 0)

green → rgb(0, 255, 0)

white → rgb(255, 255, 255)

2018 APRIL

|    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|
| M  | T  | W  | T  | F  | S  | M  | T  | W  | T  | F  | S  | S  |
|    |    |    |    |    |    | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |    |    |    |

WK 09 (061-304)

02

FRIDAY  
MARCHcolor picker can be used to select the colors  
, so not need to remember.

8 am

9 am Hex code

Hexadecimal or Hex

another format for RGB

10 am

11 am Letters: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

12 noon

e.g. color: #FF FF FF;  
R G B

1 pm

2 pm

This is based on hexadecimal number systems  
which has 16 digits.

3 pm

RGB → 0-255

same Hex → 00 - ff

4 pm

5 pm

shortcut → #fff same as #ffffff

#0ff → same #00ffff

#0f0 → same #00ff00

6 pm

1. rgb(255, 255, 255) → white  
→ #ffffff or #fff

2. Yellow → rgb(255, 255, 0) → #ffff00 or #ff0

Blue → rgb(0, 0, 255) → #0000ff or #00f

2018







06

TUESDAY  
MARCH

WK 10 (065-090)

| MARCH 2018 |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|
| M          | T  | W  | T  | F  | S  | S  |
|            | 1  | 2  | 3  | 4  | 5  | 6  |
| 7          | 8  | 9  | 10 | 11 | 12 | 13 |
| 14         | 15 | 16 | 17 | 18 | 19 | 20 |
| 21         | 22 | 23 | 24 | 25 | 26 | 27 |
| 28         | 29 | 30 | 31 |    |    |    |

8 am Tent Properties :-

9 am font-align  
font-weight  
10 am tent-decoration  
line-height  
11 am letter-spacing  
font-size

1 pm Text-align :-

2 pm text-align: left or right

3 pm text-align: right or left

4 pm text-align: center

5 pm text-align: justify

6 pm font-weight :-

100 - 900 → 100, 200, 300, 400, 500, 600, 700, 800, 900

font-weight: normal // 400

font-weight: bold // 700

font-weight: 100

font-weight: bolder

font-weight: lighter

} it has relation parent tag

2018

2018 APRIL

| M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 |
| 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 |    |    |    |    |    |

WK 10 (065-090)

07

WEDNESDAY  
MARCH

tent-decoration :-

Sets the appearance of decorative lines on text like underline.

8 am

9 am

10 am

11 am

12 noon

1 pm

2 pm

3 pm

4 pm

5 pm

6 pm

tent-decoration: underline

" : overline

" : ~~line-through~~ line-through

" : none

Shapes and color can also be changed of these lines

e.g. tent-decoration-color: red;

tent-decoration-style: solid;

other styles → solid, double, dotted, dashed, wavy

or syntax can be

tent-decoration: dotted red underline;

→ output abcd → red

common usecase →

1. underline

2. line-through

3. anchor tag

Scroll back 1 page



08

THURSDAY  
MARCH

WK 10 (067-299)

MARCH 2018

| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
|    |    |    |    |    | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
| 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 |
| 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |    |    |    |    |    |    |

8 am

another way for font style →

9 am

font-family: avant grade, didot, sans-serif;

10 am

↓  
 backup family  
 fonts  
 → if not available  
 other.

11 am

12 noon

1 pm

Text-transform:-  
control capitalization of text.

2 pm

text-transform: uppercase;  
 " " - lowercase;  
 " " - capitalized;

3 pm

4 pm

5 pm

Selectors:-

6 pm

universal selector →

\* {

\* → asterisk

element selectors →

e.g. h1 {

2018

Multiple classes can be made assigned to one element in HTML using  
 e.g. <div class="new today">  
 Spaces inside class=" " →

2018 APRIL

| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
|    |    |    |    |    | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
| 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 |
| 24 | 25 | 26 | 27 | 28 | 29 | 30 |    |    |    |    |    |    |    |

WK 10 (068-297)

09

FRIDAY  
MARCH

8 am

multiple element selectors can be merged

9 am

e.g. h1, h2, p {

10 am

ID selector →

11 am

12 noon

Selects an element based on the value of the  
 element's id attribute.  
 <h1 id="myid">  
 #myid {  
 (case sensitive)  
 unique id

1 pm

2 pm

3 pm

# is imp. here

4 pm

class selector:-

5 pm

Selects an element based on their class attribute

6 pm

e.g. &lt;h1 class="a"&gt;

&lt;button class="a"&gt;

&lt;p class="a"&gt;

grouped under  
class a

.a {

or p.a {

to specific  
element.

2018



General Sibling Selector (~) - selects all & any element that are  
parent sibling of specified element.  
(between ~ and +)  
e.g. div ~ p {

10

SATURDAY  
MARCH

Wk 10 (069-298)

| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 | 31 |    |    |    |    |    |    |    |    |    |    |    |

8 am

e.g. .myclass {

9 am

10 am

Descendant Selector :-

11 am

(e.g. Select all paragraphs inside divs

12 noon

1 pm

2 pm

3 pm

4 pm

5 pm

6 pm

Adjacent

11 SUNDAY Adjacent & Sibling Combinator :- (Immediately following)

e.g. Heading 3 that comes immediately after any paragraph.

2018

2018 APRIL

| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 |    |    |    |    |    |    |    |    |    |    |    |    |

12

MONDAY  
MARCH

Wk 11 (071-294)

8 am

9 am

10 am

11 am

12 noon

1 pm

2 pm

3 pm

4 pm

5 pm

6 pm

→ sibling combinator

e.g. p + h3 {

→

<p> </p>  
<h3> </h3> → only this will be altered.  
<h3> </h3>  
<div> <h3> </h3> </div>

Child combinator :-

e.g. Selects all buttons which are direct children of spans.

2 pm

3 pm

4 pm

5 pm

6 pm

parent ← span → button {  
child selector {

e.g. <div> <p> <ul> <li> </li> </ul> </div>

Here, it can only apply to single hierarchy parent. e.g.  
div > p  
p > ul  
ul > li

→ child selector can only go upto one level, while descendant selector can go upto multiple level, hence descendant is more used.

2018



13

TUESDAY  
MARCH

MARCH 2018

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 |    |    |    |
| 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 |
| 26 | 27 | 28 | 29 | 30 | 31 |    |    |    |    |    |    |    |    |

WK 11 (072-293)

8 am **Attribute Selector :-**  
Selects elements based on the presence or value of given attribute.

10 am e.g. `input[attribute="value"]` or `input[attribute]`

11 am  
12 noon e.g. `input[type]` → will apply wherever type attribute is used

1 pm `input[type="text"]` → " " with value text is used

2 pm `a[title]` → a elements with title attribute

3 pm `ol[type="a"]` → ol elements with type attribute with value a.

There are many more syntax of MDN.

Q. `#myId { color: green }`

Or `div[id="myID"] { color: green }`

2018

2018 APRIL

M T W T F S S M T W T F S S

|   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |
|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|

WK 11 (073-293)

14

WEDNESDAY  
MARCH

**Pseudo Class :-**

A keyword added to a selector that specifies a special state of the selected elements (s).

8 am : hover

9 am : active

10 am : checked

11 am : nth-of-type()

12 noon Syntax → element: pseudoclass {

element: hover → used when mouse cursor is moved/hover through that area, then CSS will apply

element: active → when element is clicked

checked:

element: checked → used for radio & checkbox usually when checked, CSS state will change

e.g. `input[type="radio"]:checked + label {`

`or #1 check: checked }`

2018



15

THURSDAY  
MARCH

MARCH 2018

| S  | M  | T  | W  | T  | F  | S  |
|----|----|----|----|----|----|----|
|    | 1  | 2  | 3  | 4  | 5  | 6  |
| 7  | 8  | 9  | 10 | 11 | 12 | 13 |
| 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 | 25 | 26 | 27 |
| 28 | 29 | 30 | 31 |    |    |    |

element: nth-of-type(n) → with alter ~~n~~ <sup>n<sup>th</sup></sup> element.

e.g. div: nth-of-type(2n) {

every even no. div will change

div: nth-of-type(3) {

→ 3rd div will alter

→ More pseudo-class on MDN.

Pseudo Element :-

A keyword added to a selector that lets you style a specific part of the selected element(n).

:: first-letter

:: first-line

:: selection

→ after selected letter/line/pt. or when ~~word~~ through cursor line is selected, CSS will be applied.

→ More pseudo on MDN

2018

2018 APRIL

| S  | M  | T  | W  | T  | F  | S  |
|----|----|----|----|----|----|----|
|    | 1  | 2  | 3  | 4  | 5  | 6  |
| 7  | 8  | 9  | 10 | 11 | 12 | 13 |
| 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 | 25 | 26 | 27 |
| 28 | 29 | 30 |    |    |    |    |

16

FRIDAY  
MARCH

CSS: Cascading Style Sheets

What it cascade in CSS :-

The CSS cascade algorithm's job is to select CSS declarations in order to determine the correct values for CSS properties.

e.g. h2 {

color: yellow;

h2 {

color: blue;

Final color is blue since algo. works from top to bottom & bottom is prioritized.

This also applies in ~~html~~ html, when multiple CSS are linked to html file then last CSS file added is prioritized.

Selector Specificity :-

Specificity is an algo that calculates the weight that is applied to given CSS declaration.

2018



17

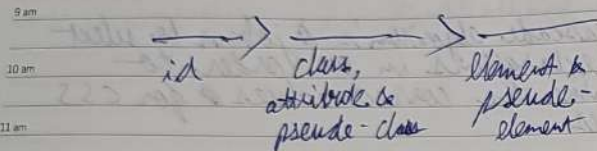
SATURDAY  
MARCH

MARCH 2018

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| M  | T  | W  | T  | F  | S  | S  |
|    |    |    | 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  | 9  | 10 | 11 |
| 12 | 13 | 14 | 15 | 16 | 17 | 18 |
| 19 | 20 | 21 | 22 | 23 | 24 | 25 |
| 26 | 27 | 28 | 29 | 30 | 31 |    |

WK 11 (076-283)

8 am 3 digit no. :-



12 noon → { id > class > element } (more priority)

2 pm e.g. h2 {

3 pm }

4 pm 0 0 1 → 1

5 pm .myclass : hover (1 class, 1 pseudo class)

6 pm 0 2 0 → 20

→ more selectors &gt; less selectors

18 SUNDAY

→ When specificity is same we use cascading.

Inline Specificity :-

Inline styles are more specific than id

2018

2018 APRIL

MTWTFSSMTWTFSS

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| M  | T  | W  | T  | F  | S  | S  |
|    |    |    | 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  | 9  | 10 | 11 |
| 12 | 13 | 14 | 15 | 16 | 17 | 18 |
| 19 | 20 | 21 | 22 | 23 | 24 | 25 |
| 26 | 27 | 28 | 29 | 30 |    |    |

WK 12 (078-287)

19

MONDAY  
MARCH

Inline Specificity

8 am

9 am

inline style

id

class, attribute &amp; pseudo-class

element &amp; pseudo-element

10 am

11 am

! important :-

12 noon

To show the most specific in document or most prioritized

1 pm

e.g. h2 {

@ color : red

! important ;

with never change

2 pm

{

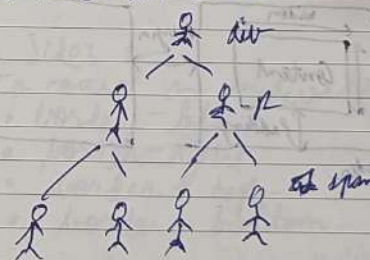
3 pm

4 pm

Inheritance in CSS :-

5 pm

6 pm



e.g. &lt;div&gt;

&lt;p&gt;

&lt;ul&gt;&lt;/ul&gt;

&lt;p&gt;

&lt;/div&gt;

Here is div is altered than all inherited child will also changed.

2018

APR



20

TUESDAY  
MARCH

WK 12 (079-285)

| MARCH 2018 |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|
| M          | T  | W  | T  | F  | S  | S  |
|            | 1  | 2  | 3  | 4  | 5  | 6  |
| 7          | 8  | 9  | 10 | 11 | 12 | 13 |
| 14         | 15 | 16 | 17 | 18 | 19 | 20 |
| 21         | 22 | 23 | 24 | 25 | 26 | 27 |
| 28         | 29 | 30 | 31 |    |    |    |

8 am → By default input boxes and buttons will not inherit its parents property.

To inherit -

e.g.

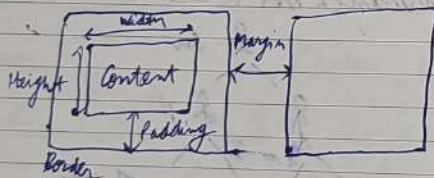
```
input {
  background-color: inherit;
}
```

12 noon → Color palette can be used from online to get good color combinations for websites.

2 pm → Some non-inheritance properties - (will not inherit to any child)  
e.g. width, height, border, etc.

4 pm **Box Model :-**

- Height
- Width
- Border
- Padding
- Margin



Height & Width

By default, it sets the content area height, of the element.

e.g.

```
div {
```

```
  height: 100px; width: 400px;
```

2018

2018 APRIL

| M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|
|    | 1  | 2  | 3  | 4  | 5  | 6  |
| 7  | 8  | 9  | 10 | 11 | 12 | 13 |
| 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 | 25 | 26 | 27 |
| 28 | 29 | 30 |    |    |    |    |

WK 12 (080-285)

21

WEDNESDAY  
MARCH

**Border :-**

Used to set an element's border

- border-width
- border-style
- border-color

e.g.

```
div {
```

```
  border-width: 5px;
  border-style: solid;
  border-color: black;
```

**Border Shorthand :-**

width (style) color

(more used)

e.g.

```
div {
```

```
  border: 2px solid blue;
```

**Border Sides :-**

To control an individual side of the box.

- border-left
- border-right
- border-top
- border-bottom

e.g.

```
border-left-color: blue;
border-left-style: dashed;
```

```
border-top: 5px dashed blue;
```

2018

APR



22

THURSDAY  
MARCH

|    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|
| M  | T  | W  | T  | F  | S  | M  | T  | W  | T  | F  | S  | S  |
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 |    |    |
| 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 | 29 | 30 | 31 |    |    |    |    |    |    |

Wk 12 (082-286)

## Border Radius :-

Used to round the corners of an element's outer border edge.

e.g.

div {

border-radius: 15px;

}

or div {

border-radius: 50%;

}

all together → border-radius: 15px 15px 25px 25px;  
individual corner :- (top; right; bottom; down;)

e.g.

div {

border-top-right-radius: 30px;

}

## Padding :-

• padding-left

• padding-right

• padding-top

• padding-bottom

e.g. div {

padding-left: 50px;

}

## Padding Shorthand :-

1. all four sides

padding: 50px;

2018

2018 APRIL

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |    |    |    |    |    |    |
| 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 |
| 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |    |    |    |    |    |    |

Wk 12 (082-283)

23

FRIDAY  
MARCH

## 2. top &amp; bottom | left &amp; right

padding: 1px 2px;

## 3. top | left &amp; right | bottom

padding: 1px 2px 3px;

## 4. top | right | bottom | left (\* clockwise)

padding: 1px 2px 3px 4px;

## Margin :-

To set spacing on the outside of border

• margin-left

• margin-right

• margin-top

• margin-bottom

e.g. div {

margin-left: 50px;

}

## Margin Shorthand :- (Same as Padding shorthand)

For elements with default margin and to set it to 0 :-

margin: 0;

2018



24

SATURDAY  
MARCH

MARCH 2018

|    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|
| M  | T  | W  | T  | F  | S  | M  | T  | W  | T  | F  | S  |
|    |    |    |    |    |    | 1  | 2  | 3  | 4  | 5  | 6  |
| 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 |
| 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |    |    |    |    |

Wk 12 (083-262)

8 am

Display :-

9 am

It sets whether whether an element is treated as a block or inline element and the layout used for its children.

10 am

11 am

display: inline

12 noon

display: block

1 pm

→ more on readn.

2 pm

3 pm

4 pm

5 pm

6 pm

display: inline block; :-

elements style block behave inline.  
(in single line)

25 SUNDAY

display: none; :-

Element will not display and hide on screen.

Relative Units in CSS :-

1. em

rem

(most used)

vw

vh

2018

vw/vh → 1% of viewport width/height. 100vh → Full height of screen.  
viewport is user's visible area of webpage. 100vw → "width"

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 |
| 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |    |    |    |    |    |    |

Wk 13 (085-280)

26

MONDAY  
MARCH

Percentage (%) :-

8 am

9 am

It is often used to define a size as relative to an element's parent object.

10 am

e.g.

e.g. width: 30%; → relative to parent

11 am

12 noon

margin-left: 50%; → relative to parent size

1 pm

Em :-

2 pm

Relative to font size of the parent, in case of typographical properties like (font-size).

3 pm

and font size of element itself, in case of other properties like width, padding, margin, etc.

4 pm

5 pm

6 pm

e.g.

#inner {

font-size: 2em; → 2 times font size of parent  
padding: 0.5em; → 0.5 times of font size of same element.

Drawback of Em :-

Snowball effect in nested elements.

To overcome the Em is introduced.

Rem (Root Em) :-

Relative to font size of the root element.

: rem;

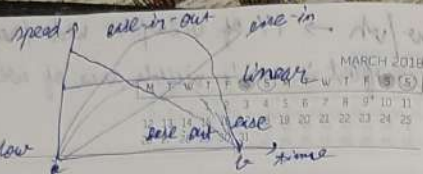
2018



## Transition timing function:-

27  
TUESDAY  
MARCH

1. Linear  $\rightarrow$  same speed
2. Ease  $\rightarrow$  slow, fast, slow
3. Ease-in  $\rightarrow$  slow, fast
4. Ease-out  $\rightarrow$  fast, slow
5. Ease-in-out  $\rightarrow$  slow, fast, slow



## Alpha Channel:-

Sets the opacity for a color.

Ranges from 0 to 1.

e.g. `rgba(255, 255, 255, 0.3)`

0  $\xrightarrow{0.5}$  1  
increase

in hex code

in hex code  $\rightarrow$  00  $\rightarrow$  ff  
(hidden) (full)

e.g. `#ff0000`

## Opacity:-

Sets the opacity for the element.

Ranges from 0 to 1.

e.g. `opacity: 0.5;`

## Transitions in CSS:-

Transitions enable you to define the transition between two states of an element.

e.g. `div:hover {`

`background-color: black;`  
`border-radius: 50%;`

`div { height: 200px;`  
`width: 200px;`  
`background-color: green;`  
`transition: 1s;`

Now if we hover it with 1s to fully change

6. Step-start  $\rightarrow$  steps(4, start)
7. Step-end  $\rightarrow$  steps(4, end)
8. Cubic-bezier  $\rightarrow$  (a, b, c, d) (custom)

2018 APRIL

M T W T F S S

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

Wk 13 (187-278)

28  
WEDNESDAY  
MARCH

## Transition Shorthand:-

property name / duration / time-function / delay.

e.g. `transition: margin-top 2s ease-in-out 0.2s;`

`transition-duration: 1s;` same  
`transition-property: 1s;`

`transition-delay: 0.2s;`  $\rightarrow$  delay before transition

time-function  $\rightarrow$  `ease-in`

$\rightarrow$  `margin: 0 auto;`  $\rightarrow$  sets equal margin from left and right. (no element will be in center)

## CSS transform:-

This property lets you rotate, scale, skew, or translate an element.

## 1. Rotate:-

e.g. `transform: rotate(45deg);`

$\rightarrow$  will rotate by 45 degree clockwise

deg  $\rightarrow$  degree  
rad  $\rightarrow$  radian  
turns  $\rightarrow$  turns  
more on mdn.

APR

2018



29

THURSDAY  
MARCH

WK 13 (088-277)

MARCH 2018

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 | 31 |    |    |    |    |    |    |    |    |    |    |    |

8 am

2. Scale :-

e.g. transform: scale (1.5);  
↳ becomes 1.5 times

9 am

transform: scale (0.5, 2);  
↳ 0.5 times width  
2 times height

10 am

11 am

transform: scaleX (2); 2 times width

12 noon

transform: scaleY (0.5); 0.5 times height

1 pm

2 pm

3. Translate :-

e.g. transform: translate (50px, 50px);  
↳ moves 50px X axis (right)  
and 50px Y axis (down)  
(-ve sign for opposite)

3 pm

4 pm

transform: translateX (10px);

5 pm

transform: translateY (20px);

6 pm

4. Skew :-

e.g. transform: skew (30 deg);  
↳ tilts by 30 deg

transform: skew (-90 deg);  
↳ tilts 90 deg and it will vanish at this deg.

→ All these properties is also applied to the content also.

2018

2018 APRIL

M T W T F S S M T W T F S S

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 | 31 |    |    |    |    |    |    |    |    |    |    |    |

WK 13 (088-278)

30

FRIDAY  
MARCH

8 am

9 am

10 am

11 am

12 noon

1 pm

2 pm

3 pm

4 pm

5 pm

6 pm

(transform shorthand)  
→ All the transform properties can also be applied in single line.

e.g. transform: rotate(30deg) translate(10px, 2px) scale(2) skew(20deg);  
↳ 30 degree rotate, 10px shift in X axis  
2px shift in Y axis, 2x → becomes double, skew (tilt) by 20 degree.

Box Shadow :-

It adds shadow effects around an element's frame

e.g. box-shadow: 2px 2px 10px green;  
↓ ↓ ↓ ↓  
X offset Y offset blur shadow color

Background Image :-

Let's you set an image as a background.

background-image: url ("bgimage.jpg");

↳ the ~~big~~ ~~large~~ bgimage may be zoomed in due to difference in sizes.  
So we use :-

background-size: contain / cover / auto;

background-position

- repeat

- clip

may repeat  
no fillcrop to fill  
(best)

stretch to fill

2018



31

SATURDAY  
MARCH

WKL 13 (1990-279)

MARCH 2018

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 | 31 |    |    |    |    |    |    |    |    |    |    |    |

8 am Position:-

The position CSS property sets how an element is positioned in a document.

The top, right, bottom and left properties determine the final location of positioned elements.

- static
- relative
- absolute
- fixed

Position - Static:-

The top, right, bottom, left, 2-axis properties have no effect.

This is default value position.

position: static;

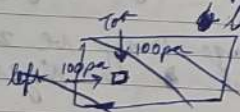
Position - Relative:-

The offset is relative to itself based on the values of top, right, bottom and left.

position: relative;

top: 100px;

left: 100px;



Pos-Absolute:-

The element is removed from the normal document flow, and no space is created for the element in the page layout.

It is positioned relative to its closest positioned ancestor, if any; otherwise it is placed to the initial containing block or body. position: absolute;

MARCH 2018

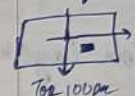
MAY 2018

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| M  | T  | W  | T  | F  | S  | S  |
| 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 |
| 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 | 31 |    |    |    |    |

MON TUE WED THU FRI SAT SUN

Relative Position:- (center it itself)

Before

top: 100px  
left: 100px

left: 100px

Top: 100px

Absolute Position:- Center point in top-left corner of its parent or (if no parent) then body (whole doc.)

top: 50px  
left: 50px

50px  
or parent does not have position.

It does not have any space (no margin overlap).

A new element can take its original position as it does not exist.

Position-Fixed:- Same as absolute but does not depend on parent, means only and only body whole body. And it will always at its position even while scrolling while absolute is fixed in document.

position: fixed;

(some exceptions are there -> read MdN)

Position-Sticky:- -&gt; Read MdN.

→ cursor:

Cursor property sets the mouse element, a cursor changes when it is over the element.

e.g cursor: pointer; help; zoom-in; grab; wait; etc.

APR



01

SUNDAY  
APRIL

APRIL 2018

| M  | T  | W  | T  | F  | S  | M  | T  | W  | T  | F  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|
|    |    |    |    |    |    | 1  | 2  | 3  | 4  | 5  | 6  |
| 8  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |    |    |

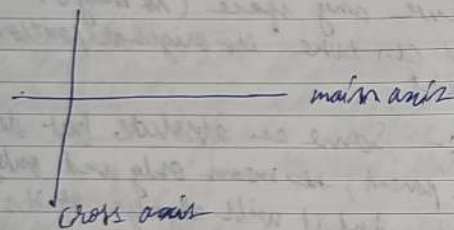
WK 13 (091-274)

8 am Flexbox :-  
Flexible box layout

9 am It is a one-dimensional layout method for  
arranging items in rows or columns.

10 am It ~~may~~ helps make website to make responsive  
(e.g. change of screen, hence need flexible layout)

11 am The Flex Model :-



direction = row

5 pm e.g.



display: flex

Flexbox Direction :-

Sometimes items inside a container (e.g. div inside div) ~~take~~ takes extra space than container (or parent).  
To ~~tack~~ tackle this we use flexbox.

2018 MAY

| M  | T  | W  | T  | F  | S  | M  | T  | W  | T  | F  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 |
| 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 |
| 26 | 27 | 28 | 29 | 30 | 31 |    |    |    |    |    |    |

WK 14 (092-273)

02

MONDAY  
APRIL

display: flex; → defaultly fit the item in row  
↓ same ~~into~~ inside a container.

default flex-direction: row;

flex-direction: column; → fit the items in a  
column inside container.

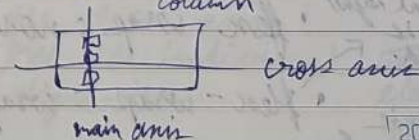
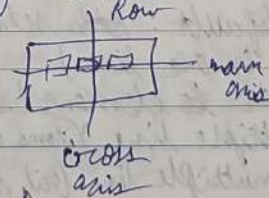
flex ~~fit~~ fits the items inside container despite  
is default/given width or height (ignores ~~changes~~  
own height & width)

Flexbox Direction :-

It sets how flex items are placed in a the  
flex container, along ~~which axis~~ ~~with~~ which  
axis and direction

- flexbox-direction: row; main axis, left to right
- flexbox-direction: row-reverse; " , right to left
- flexbox-direction: column; " , top to bottom
- flexbox-direction: column-reverse; " , bottom to top.

main axis is based on direction. So opposite  
of main axis is ~~not~~ cross axis.



2018



03

TUESDAY  
APRIL

APRIL 2018

| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
|    |    |    |    |    | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |    |
| 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 |
| 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |    |    |    |    |    |    |

WK 34 (2018-272)

8 am

Justify Content :-

Tells how the browser distributes space between and around content items along the main-axis. (depends on direction)

9 am

10 am

11 am

12 noon

1 pm

2 pm

3 pm

4 pm

5 pm

6 pm

- justify-content: flex-start; (default)

- justify-content: flex-end;

- justify-content: ~~flex~~ center;

First/last element in controlled by direction  
Space in the container is controlled by justify-content (also depends on direction)

- justify-content: space-between; → sets equal space to fill container b/w items. (no space in right & left)

- justify-content: space-around; // (same) (half space L & R than item's space)

- justify-content: space-evenly; // (same) (full space L & R equals item's space)

Flex Wrap :-

Sets whether flex items are forced onto one line or can wrap onto multiple lines.

Original height  
width is fixed

- flex-wrap: nowrap; (default: sets elements in one line)

- flex-wrap: wrap; (sets to element/items in multiple line items are (by cross axis))

Cross axis  
Reverse  
dir="reverse"

- flex-wrap: wrap-reverse; (multiple line but in reverse)

2018 MAY

| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 |    |
| 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 |
| 28 | 29 | 30 |    |    |    |    |    |    |    |    |    |    |    |

(all before are mostly main axis based)

04

WEDNESDAY  
APRIL

Align Items :-

8 am

9 am

10 am

11 am

12 noon

1 pm

2 pm

3 pm

4 pm

5 pm

6 pm

Distributes our items along the cross axis. (space should be available in cross axis)

(Default) • align-items: flex-start; } not text based, rather element based.

• align-items: flex-end;

• align-items: center;

• align-items: baseline; → text-based (content will be at same height)

Align Content :-

It sets the distribution of space between and around content items along a flexbox's cross axis.

(Same as justify-content but in cross axis)

- align-content: ~~flex-start~~ flex-start / flex-end / center

- align-content: space-between / space-around / evenly

- align-content: baseline;

Align Self :-

Aligns an item along the cross axis. It is same as align items but for individual item.

• align-self > ~~flex-start~~ align-items

more priority

2018



05

THURSDAY  
APRIL

WK 14 (095-270)

| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |    |    |    |    |    |    |
| 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 |
| 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |    |    |    |    |    |    |

8 am • align-self: flex-start / flex-end / flex-center;

9 am • align-self: baseline;

10 am Flex Sizing: (grow / shrink)

11 am • flex-basis (along main-axis)  
It sets the initial main size of a flex item. (should have enough space in the container)  
e.g. flex-basis: 100px; (also depends on font size)

e.g.



4 pm • flex-grow  
It specifies how much of the flex container's remaining space should be assigned to the flex item's main size.

5 pm e.g. flex-grow: 1; (default)  
It grows in proportion  
will get half

From → flex-grow: 1 From → flex-grow: 1

will get equally (half/half)

2018

Flexbox games → [codingfantasy.com](http://codingfantasy.com)

2018 MAY

| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 | 31 |    |    |    |    |    |    |    |    |    |    |    |

WK 14 (095-259)

06

FRIDAY  
APRIL

→ max-width:

→ min-width:

8 am

} width will not increase/decrease more than this.

9 am

• flex-shrink

It sets the flex factor of flex item. (if items are over grown) (default)

flex-shrink: 1;

flex-shrink: 2; → will shrink two times

12 noon

1 pm

Flex Shorthand :-

2 pm

• flex-grow / flex-shrink / flex-basis

e.g. flex: 2 2 100px;

3 pm

4 pm

• flex-grow / flex-basis  
e.g. flex: 2 100px;

5 pm

• flex-grow (unless)

e.g. flex: 2;

6 pm

• flex-basis

e.g. flex: 100px;

→ flex-grow, flex-shrink depends on flex-basis, or change corresponding flex basis also.

e.g.

item 1: flex: 1 50px; item 2: flex: 1 100px;  
container → 300px

item 1 + item 2 = 150px (150px left)  
(initial) (both will get half/half extra space left i.e. 75/75px)

item 1 → 50 + 75 = 125

item 2 → 100 + 75 = 175

2018



07

SATURDAY  
APRIL

In maximum case flex is used than grid.  
Flex & grid, if applied to any element then it  
will only apply to direct child (not sub-children or grand-children)  
Parent → Container, Direct Childs → Items

APRIL 2018

8 am → Binary Search Queue

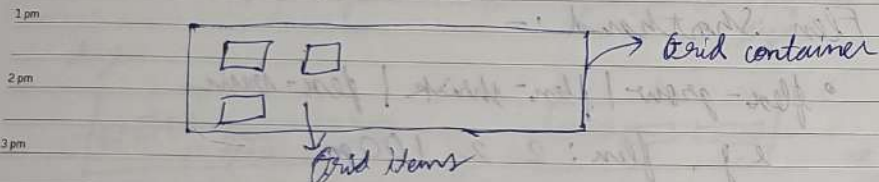
9 am CSS Grid :-

10 am Setting a container's display to grid will make all  
children grid items.

11 am

12 noon

e.g. container {  
display: grid;



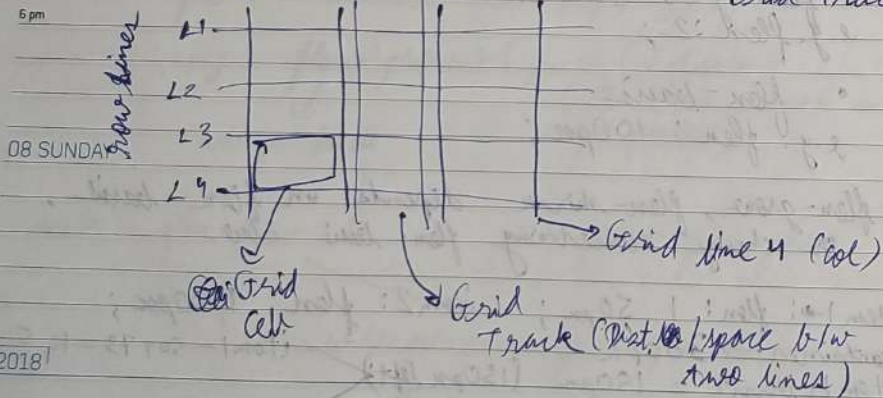
Grid Model :-

column lines

Grid lines

Grid cell

Grid Track



Grid is hard to use and understand due to its discrepancy lies.  
So when to use grid? Only when row and column both are  
needed.  
If 1D is only use case then use Flexbox.

09

MONDAY  
APRIL

Grid Template :-

They define the line &amp; track sizing.

display: grid; → default rows will be made as many  
items are in the container (may overflow)

For row wise

track(n) = row lines - 1

For col wise

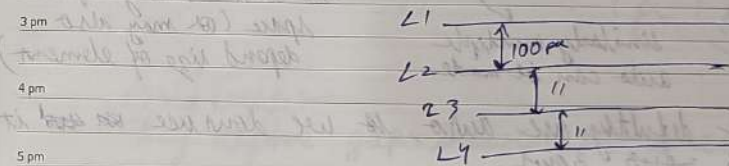
track(n) = col lines - 1

• grid-template: rows;

e.g.

11 : 100px 100px 100px;

→ 3 grid tracks



If space is left then other  
items are distributed equally.

If items size are bigger than row-track a row-grid  
size the items will take row-track according  
to their size. If there is no extra items then  
then there will be no extra track  
if space is left with no items.

• grid-template-columns:

e.g. 11 : 100px 100px 100px;

2018

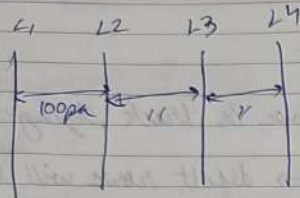


10

TUESDAY  
APRIL

WK 15 (100-265)

| APRIL 2018 |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|
| M          | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|            |    |    |    |    | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |    |
| 9          | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 |
| 23         | 24 | 25 | 26 | 27 | 28 | 29 | 30 |    |    |    |    |    |    |



All items are placed such that it fills the column first then moves to next row.  
If items are extra then grid always create extra rows, not columns.

Grid-template-columns: auto;  
: auto auto;  
→ 2 column taking all space  
→ 2 column taking equal space (may also depend size of element)  
Similarly multiple auto can be made.

rows defaultly use auto, so we don't use ~~auto~~ it with ~~rows~~ rows.

auto depends on element size and also content in it, hence we will not use auto much.

If there is no height/width of the items then items will ~~not~~ acquire full grid cell for themselves.

• Repeat is used to divide all available space.

- grid-template-rows: (count, 1fr);
  - grid-template-columns: (count, 1fr);
- repeat

2018 MAY

| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 | 31 |    |    |    |    |    |    |    |    |    |    |    |

11

WEDNESDAY  
APRIL

row count

WK 15 (101-264)

(better than auto)

- grid-template-rows: repeat(count, 1fr);
- grid-template-columns: repeat(count, 1fr);

columns count

1fr → 1 fraction of 100% available space

width for rows  
height for cols.

(extra items will get less space)  
↳ (in rows)

(another way)

- e.g. grid-template-rows: repeat(2, 1fr);
- 11 : 1fr 1fr; (same)  
→ 2 times

Grid Gaps:

They define gap between the lines.

- row-gap: ; e.g. 10px;
- column-gap: ;
- grid-gap: (row-gap) (col-gap); (short-hand)

2018



12

THURSDAY  
APRIL

Wk 15 (102-263)

| APRIL 2018 |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|
| M          | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|            |    |    |    |    | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |    |
| 9          | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 |
| 23         | 24 | 25 | 26 | 27 | 28 | 29 | 30 |    |    |    |    |    |    |

8 am

Grid-columns :-

Defines an item's starting &amp; ending positions inside the columns.

9 am

10 am

11 am

12 noon

1 pm

2 pm

3 pm

4 pm

5 pm

6 pm

• grid-column-start : line-no. ;

• grid-column-end : line-no. ;

e.g.

" - start : 1 ;

" - end : 3 ;

> item will take space from line 1 to line 3, which means 2 grid tracks.

It reserves the allotted columns even if individual size of item is lesser (then reserved space will be empty).

short-hand -

• grid-column : start\_col / end\_col ;

or

• grid-column : start\_col / span no. ;

↳ e.g. grid-column : ~~start~~ 1 / span 3 ;

↳ will take 3 space/tracks after line 1.

↳ This is same as  
grid-column : 1 / 4 ;

It is better to apply grid-columns / grid-rows on individual or set of items.

| 2018 MAY |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|
| M        | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
| 1        | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15       | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29       | 30 | 31 |    |    |    |    |    |    |    |    |    |    |    |

Wk 15 (103-262)

13

FRIDAY  
APRIL

8 am

9 am

10 am

11 am

12 noon

1 pm

2 pm

3 pm

4 pm

5 pm

6 pm

Grid-Rows

Defines an item's starting &amp; ending position inside the row.

(every thing same as grid-column)

• grid-row-start : line-number

• grid-row-end : line-number  
(short-hand -)• grid-row : start\_row / end\_row ;  
or

• grid-row : start\_row / span no. ;

Grid-row takes space of allotted rows as well as side columns. Means full row.  
Whereas column only took allotted columns, not full columns.

Grid Common Properties :-

- |                             |                              |                                   |
|-----------------------------|------------------------------|-----------------------------------|
| • justify-items (container) | } Horizontal                 | } start / end / center ;<br>OR OR |
| • justify-self (item)       |                              |                                   |
| • align-items (container)   | } Vertical                   |                                   |
| • align-self (item)         |                              |                                   |
| • place-items (container)   | } Both horizontal & vertical | 1/                                |
| • place-self (item)         |                              |                                   |



14

SATURDAY  
APRIL

WK 15 (104-261)

APRIL 2018

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|    |    |    |    |    |    |    | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 |
| 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |    |    |    |    |    |

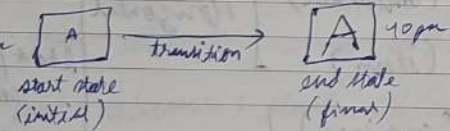
8 am ⇒ ~~display~~ display: flex; } Both are block elements  
 9 am ~~display~~ display: grid; } (or creates the element block element)

To make them inline :-

display: inline-flex; } Both are now inline  
 display: inline-grid;

2 pm CSS Animations :- (similar to transition but gives <sup>even</sup> extra control on items)  
 To animate CSS elements (using name)

3 pm e.g. @keyframes myName {  
 4 pm 0% → from { font-size: 20px; }  
 100% → to { font-size: 40px; } } Just template blue print.



15 SUNDAY  
 div {  
 animation-name: myName;  
 animation-duration: 4s;

how anim animation will be applied

7 • animation-fill-mode

(none; default) forwards; backwards;

8 • animation-play-state

(running; paused)

16

MONDAY  
APRIL

WK 16 (106-259)

2018 MAY

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 | 31 |    |    |    |    |    |    |    |    |    |    |    |

1 • animation-name

2 • animation-duration

3 • animation-timing-function

same as in transition

4 • animation-delay

5 • animation-iteration-count

e.g. (1; 2; 3; infinite;)

6 • animation-direction

(normal; reverse; alternate; alternate-re;)

Animation Shorthand :-

animation: name | duration | timing-function | delay | iteration-count | direction;

e.g. animation: myName 2s linear 3s infinite normal;

in Animation :-

@keyframes myName {  
 0% { color: red; }  
 50% { color: green; }  
 100% { color: blue; }  
 }

More about animation on MDN.

2018



17

TUESDAY  
APRIL

WK 16 (107-258)

| APRIL 2018 |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|
| M          | T  | W  | T  | F  | S  | S  |
|            |    |    |    |    | 1  | 2  |
| 3          | 4  | 5  | 6  | 7  | 8  | 9  |
| 10         | 11 | 12 | 13 | 14 | 15 | 16 |
| 17         | 18 | 19 | 20 | 21 | 22 | 23 |
| 24         | 25 | 26 | 27 | 28 | 29 | 30 |

8 am

Media Queries :-

9 am

Help create a responsive website.

10 am

Every device has its own environment or media features

11 am

- Width
- Height
- Orientation (Landscape/Portrait) (many more)

12 noon

Media queries is used to create responsive changes according to/in accordance with media features/properties.

1 pm

2 pm

- media features - width (of viewport)

3 pm

```

→ @media (width: 400px) {
  div {
    color: red;
  }
}

```

→ change only and exactly at width. (not so much used)

4 pm

5 pm

6 pm

```

→ @media (min-width: 400px) {
  div {
    background-color: red;
  }
}

```

(most used)  
change above, width  $\geq 400px$

```

→ @media (max-width: 400px) {
  div {
    background-color: red;
  }
}

```

(most used)  
change below, width  $\leq 400px$

2018

2018 MAY

| M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 |
| 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 | 31 |    |    |    |    |

WK 16 (108-257)

18

WEDNESDAY  
APRIL

8 am

```

→ @media (min-width: 200px) and (max-width: 300px) {
  div {
    background-color: red;
  }
}

```

(most used)

9 am

10 am

- orientation (of viewport)

11 am

```

@media (orientation: landscape) {
  div {
    color: red;
  }
}

```

12 noon

1 pm

2 pm

3 pm

Media Queries → more on Mon.

4 pm

Z-index :-

It decides the stack level of elements.

5 pm

Overlapping elements with a larger z-index cover those with a smaller one.

6 pm

- z-index: auto (0) → default
- z-index: 1 / 2 / ...
- z-index: -1 / -2 / ...

Default z-index of every element is 0.  
z-index won't apply to default or static positioning.  
z-index applies position: relative; or absolute; ... more.

2018



19

THURSDAY  
APRIL

WK 16 (109-256)

| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |    |    |    |    |    |    |
| 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 |
| 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |    |    |    |    |    |    |

(UI/UX)

## Basic Design Principles :-

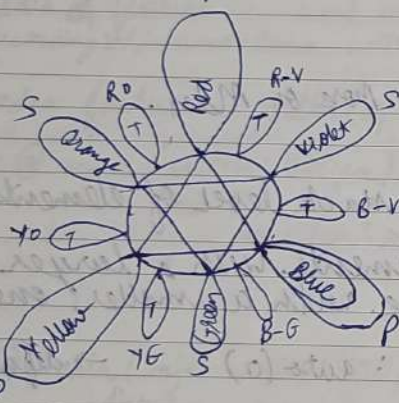
For better looking websites.

→ Color

→ Typography

## Color Theory :-

Color Wheel - P



P - Primary  
S - Secondary  
T - Tertiary

## Meaning of colors -

(Google)

[There is no fixed rule  
but we can look up for it]

APRIL 2018

2018 MAY

20

FRIDAY  
APRIL

WK 16 (110-255)

| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 | 31 |    |    |    |    |    |    |    |    |    |    |    |

## combination of colors -

- Subtle Color Combination - Monochromatic Palettes
- Impactful Color Combination - Complementary "
- High Contrast Combination - Triadic "

(Learn more of canvas → color wheel)

## Typography :-

Style &amp; Appearance of Text

## Font Families -

- sans-serif
- serif
- monospace
- cursive
- fantasy

→ Use maximum two font style/family  
in single section

## Google Fonts -

Library of fonts (free) (1513 fonts)  
fonts.google.com

## Icons :-

[Google Icons  
Font Awesome] Free

→ dplyr.com (For links)

→ copy link in head and code in body

&lt;i&gt; &lt;/i&gt; &lt;i&gt; &lt;/i&gt; → icon HTML

2018



21

SATURDAY  
APRIL

WK 16 (111-254)

visit  
(bootstrap.com)APRIL 2018  
M T W T F S S M T W T F S S  
1 2 3 4 5 6 7 8  
9 10 11 12 13 14 15 16 17 18 19 20 21 22  
23 24 25 26 27 28 29 30

8 am

9 am

10 am

11 am

12 noon

1 pm

2 pm

3 pm

4 pm

5 pm

6 pm

Bootstrap :- (framework - Toolkit) (Pre-built CSS)  
(framework) (stylebook)

Bootstrap is a powerful, feature-packed frontend toolkit. Build anything - from prototype to production - in minutes (fast).

(It works on the concept of flexbox)

Benefits of Bootstrap :-

- Makes development Faster & Easier
- Good choice of pre-built components (responsive)
- Good browser compatibility.
- Two ways to use bootstrap, either download it or use link (cdn)
- cdn - Content Delivery Network  
It will make sure content is delivered from nearest server, to faster the user experience / decrease loading time.

22 SUNDAY

- Javascript file (pre-built) can also be included using link before `</body>` ~~under~~ inside `<script>` tag.  
(covered in course) `</script>`

&gt; Components

pre-built & reusable piece of user interface (UI)  
e.g. button, forms

Layouts

Overall structure and arrangement of the elements  
e.g. container, grid

Bootstrap can save lots of time to CSS codes.

2018 MAY

M T W T F S S M T W T F S S

1 2 3 4 5 6 7 8 9 10 11 12 13  
14 15 16 17 18 19 20 21 22 23 24 25 26 27  
28 29 30 31

WK 17 (113-252)

23

MONDAY  
APRIL

Components :-

Containers :-

(Layout)

Containers are used to contain, pad and (sometimes) center the content within them.

class = "container" → (more on bootstrap)

- Bootstrap even changes font-style, font-family, font-weight, etc.

Components :-

Buttons :- (most useful)

e.g. `[Submit]`

button

(base class)

→ more on bootstrap

class = "btn"

Button can be disabled using disabled inside button tag.  
e.g. `<button class="btn" disabled>Button</button>`  
anchor tags & input

change any element to button →

e.g. `<a href="#" class="btn">`In `<a>` role = "button" should be added for screen-readers.

Badges :-

e.g. `[Notifications 10+]` → badge

It is mostly created in a span

e.g. `<span class="badge">New</span>`We can change badge's position, size, b-g colors and more.  
→ more on bootstrap.

It can also be added inside buttons by including span.

2018



If we link a CSS file after bootstrap link, CSS file will override styles if changed.

24

TUESDAY  
APRIL

Wk 17 (114-250)

| M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|
|    |    |    |    |    |    |    |
| 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 |
| 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 | 31 |    |    |    |    |

8 am Alert :- e.g. `[Alert - Failed!]` (can be used as popup)

9 am e.g. `<div class="alert">`

10 am Button & Group :- `> more on bootstrap`

11 am e.g. `<div class="btn-group">`  
`<button>` `<div class="left">` `</div>` `</button>`  
`<button>` `<div class="middle">` `</div>` `</button>`  
`<button>` `<div class="right">` `</div>` `</button>`  
`</div>`

12 noon checkbox, radio, etc. can also be grouped as buttons.

3 pm Navbar :- (Navigation Bar)

4 pm e.g. `[Navbar Home Price About Search Search]`

5 pm `> code on bootstrap`

6 pm Cards :- `> code on bootstrap`

Layouts :-  
 Grid (Layout) :- flexbox based grid system  
 Bootstrap grid works on 12 column system.

class = "container" may be included to use this

e.g. `<div class="container">`  
`<div class="row">`  
`<div class="col"> 1 </div>`  
`<div class="col"> 2 </div>`  
`</div>`  
 Both with take 50-50%  
 (6-6)

2018

2018 MAY

| M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|
|    |    |    |    |    |    |    |
| 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| 8  | 9  | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 |
| 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 | 31 |    |    |    |    |

Wk 17 (115-250)

25

WEDNESDAY  
APRIL

• Grid (Responsive) - Grid options

8 am

9 am

10 am

11 am

12 noon

1 pm

2 pm

3 pm

4 pm

5 pm

6 pm

`> check on bootstrap`  
 It can be used to basically to make grid more responsive for different screen sizes.  
 Many grid options can be combined/added to same div.

• Grid (column) `> (vertical alignment / cross axis)`  
 e.g. justify-content, align-items, align-self, etc (in CSS)  
`> all these works in bootstrap also`

Forms :-

• Form controls - GA controls like inputs, textareas with upgrade with custom styles, sizing, focus states and more.  
`> check on bootstrap`

• Select  
`<select class="form-select">`  
`> more on bootstrap`

• Checkbox & Radio in form  
`> check on bootstrap`

• Layout  
 Grid layout can be implemented in forms also.  
`> check on bootstrap`

2018



26

THURSDAY  
APRIL

WK 17 (116-248)

APRIL 2018

| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
|    |    |    |    |    | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |    |
| 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 |
| 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |    |    |    |    |    |    |

## Project :- Spotify (Only HTML & CSS)

First and most important thing to do ~~is~~ is identify all the boxes. Divide them in as many ~~boxes~~ `<div>`s as possible. Then approach the divs one by one.

New properties in this project :-

- `overflow`: visible - content is not clipped renders outside element box (default)
- : hidden - clipped to rest of the content in hidden
- : scroll - provide scrollbar both ~~as~~ x & y-axis even if it's not required
- : auto - provide scrollbar only if necessary.

• custom scrollbar :- (pseudo element)

```
::-webkit-scrollbar {
  width: 10px;
}
::-webkit-scrollbar-track {
  background-color: #f0f0f0;
}
::-webkit-scrollbar-thumb {
  background-color: #808080;
}
```

• 1 rem = 16 px

- If parent is display: flex; then we can give children, `flex: (value);`  
 e.g. `flex: 1;` → it will take all remaining space in the div.  
 e.g. `flex: 0.5;` → " " half of remaining space " ".

2018 MAY

| M  | T  | W  | T  | F  | S  | S  | M  | T  | W  | T  | F  | S  | S  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 |    |
| 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 |
| 28 | 29 | 30 | 31 |    |    |    |    |    |    |    |    |    |    |

WK 17 (117-248)

27

FRIDAY  
APRIL

- ~~position~~ position: sticky; } For new bar.  
 top: 0px;  
 z-index: 10;  
 → so it will be always on the top

- appearance: none; → removes most of the default styling.  
 : auto; → acts as none on element with no special styling.

- width: fit-content; → div/element only takes/wider space according to child/content in it.

- input slider (Range)

e.g. `<input type="range" min="1" max="100" value="0" step="10">`  
 increases in step of 10. → default value is '0'

appearance: none; → removes default styling

custom slider :- (pseudo element)

```
::-webkit-slider-runable-track {
  background-color: #f0f0f0;
}
::-webkit-slider-thumb {
  background-color: #808080;
}
```

## Future Upgrades :-

- Add play icon/ing on every card when hover.
- Add toggle to library menu and player options for smaller screens.
- Add JavaScript. - Add Back End.

2018



go (like add a task to todos array)

allows you to extract  
the Redux state using  
returns the data)

const redux =

(state) => state.todos);

todos list

to render dispatch  
by giving the  
value variable

latter  
tion

ures/todos/todSlice"

table  
not needed)

## Tailwind CSS :-

"Rapidly build modern websites without leaving your  
HTML"

"Utility-First" means the framework's main focus  
is on providing utility classes.

- Benefits
- reduces confusion
  - Bundle Size (only includes the classes we  
are using, nothing extra)

Cdn is not recommended for production  
level code

Refer Doc for CLI -> Using Post CSS

Can install vite to run live (auto refresh)

by - npm i vite

but have to add in package.json

```
"scripts": {  
  "start": "vite"  
},
```

- npm install
- npm i vite
- npm run tailwind init (-y)
- tailwind.config.js -> content [ "\*/" ]  
-> will apply everywhere
- style.css -> add directives
- Change in package.json for vite
- link CSS with html
- npm run start

1 in tailwind -> 0.25 rem (4px default)

Tailwind CSS resets all H1, H2, H3, ... to same size.



Typography + refer Doc Typography

Color

Spacing

Responsive Design

Square Bracket Notation

e.g.  $w - [100px]$

$h - [10vh]$

↳ custom value

↳ ~~one~~ increases bundle size

Apply Directive :-

Use @ apply to inline any existing utility classes into your own CSS. Custom CSS.

↳ To solve repetitiveness of using Tailwind classes.

e.g. In style.css

.box {

@ apply bg-purple-500 text-white m-4 border-radius;

}

↳ use at last

<div class="box">

<div class="box">

works everywhere.

Card in Tailwind CSS

Major Project :- (Assign)

Step 1 - Initialize bundle  
npm, express, ...

Step 2 -

Model: Listing:-

place Cap  
(Schema) → title  
→ description  
→ image  
→ price  
→ location  
→ count

Test it on /test  
Step 3 - ./init

Step 4 - Index Route  
GET  
(all other same)  
./views

Step 5 - ~~Route~~  
./show

Step 6 -  
GET /listing  
Submit →